

DIGITAL SYSTEM DESIGN

1. Differentiate between weighted and non-weighted binary codes. give example of each.

Ans:

Weighted codes	Non-weighted codes
1. Each position in the code has a specific weight.	1. Position of code no not have a specific weight.
2. Decimal value of code can be determined.	1. Decimal value assigned to each code.
3. Example of these codes is: BCD, 8421, 6421, 4221, 5211, 3321 etc.	1. Examples of non-weighted codes are: Unweighted BCD code, Excess-3 code and gray code.

Example of each:

BCD: 8421 Code 94 → 9 4 ↓ ↓ Code → 8+4+2+1 8+4+2+1 1+0+0+1 0+1+0+0 =1001 0100	(48)10 into Excess-3 code 48 → 4 8 +3 +3 Excess-3: 7 11 Code ↓ ↓ 0111 1011
---	--

2. Design a code converter to convert a 4-bit binary number 1101 to it's corresponding grey code. Provide the truth table and circuit diagram.

Ans:

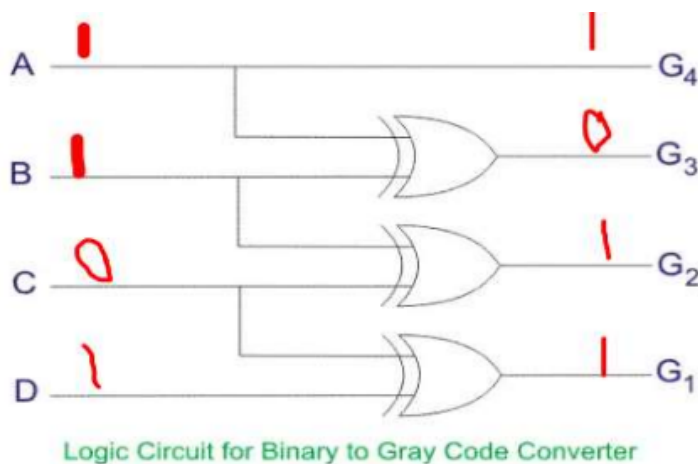
Converting a binary number to its corresponding Gray code involves a specific algorithm where each bit of the Gray code is calculated based on the corresponding bits of the binary number. Let's first create a truth table for the conversion:

Binary (B3 B2 B1 B0)	Gray (G3 G2 G1 G0)
1101	1011

Truth table for XOR gate:

B	XOR	Output
0	0	0
0	1	1
1	0	1
1	1	0

Using this truth table, you can create the XOR gates for each bit conversion as shown in the circuit diagram. The binary to gray code converter circuit is shown below:



In this circuit:

- B3, B2, B1, B0 are the input binary bits.
- B3, B2, B1, B0 are the input binary bits.
- Each pair of binary bits (B3-B2, B2-B1, B1-B0) is connected to a separate XOR gate.
- The output of each XOR gate is a corresponding Gray code bit (G3, G2, G1, G0).

3. Write the advantages of grey code over the straight binary number sequence.

Ans:

Gray code, also known as reflected binary code, offers several advantages over the straight binary number sequence:

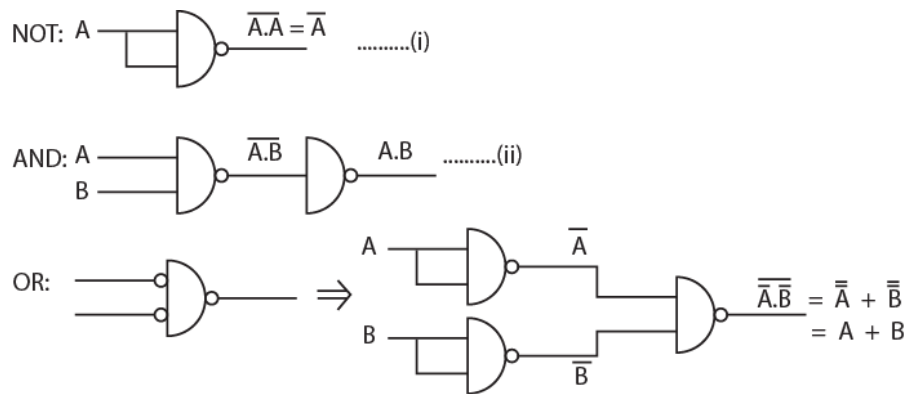
- The advantage of the Gray code over straight binary number sequence is that only one bit in the code group changes in going from one number to the next.
- Gray code ensures that only one-bit changes between consecutive values, reducing the chances of errors due to bit transitions, crucial in applications sensitive to errors, like digital communication and rotary encoders.

- Gray code is widely used in binary counters and digital control systems, as it prevents false triggering and ensures consistent counting sequence.
- Gray code prevents glitches that can arise when changing straight binary numbers due to varying propagation delays in digital systems, ensuring accurate and glitch-free transitions.

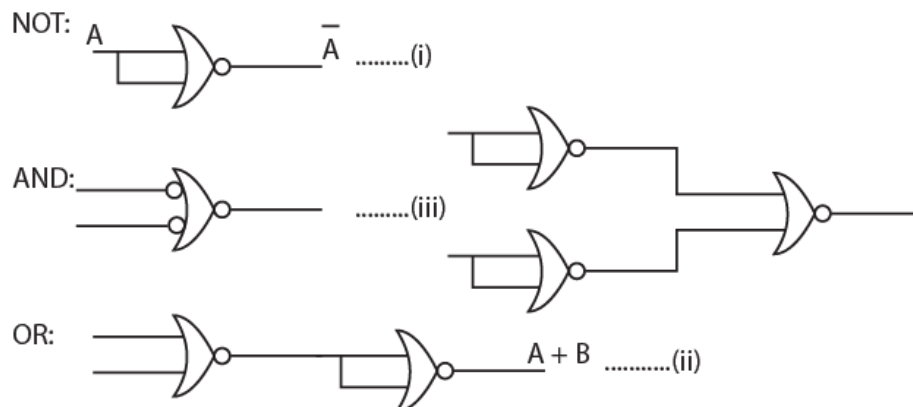
4. Why NAND gate and NOR gate are called universal gate? Explain it with proper diagram.

Ans:

In Boolean Algebra, the NAND and NOR gates are called universal gates because any digital circuit can be implemented by using any one of these two i.e. any logic gate can be created using NAND or NOR gates only. Although and, or and not are basic gates but each of their functionality can be derived using nand and not as well. One more important aspect because of this AOI logic can be converted to either Nand or Nor logic. The following is the implementation of all the gates using NAND:-



Now the following is the implementation of all the gates using NOR:-



This demonstrates how a combination of NAND gates can be used to implement different logic gates. This universality property is the reason why NAND and NOR gates are called universal gates in digital logic design.

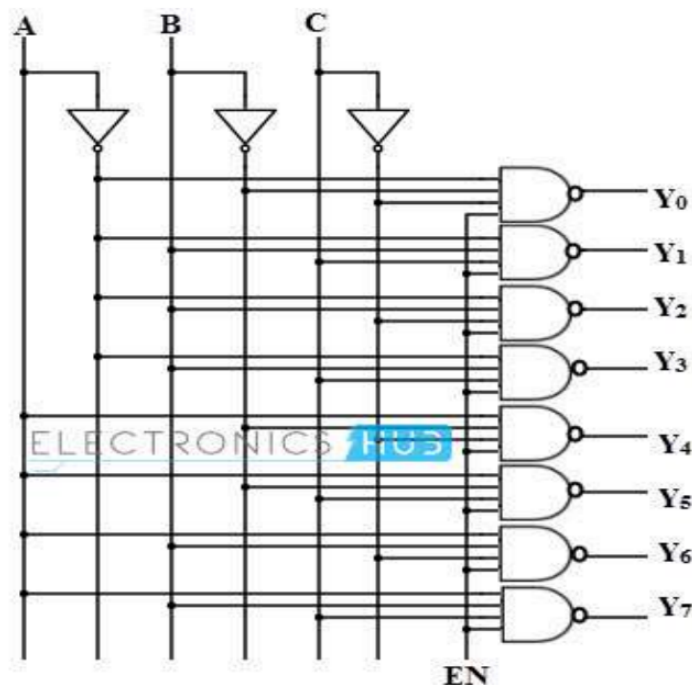
5. Construct a 3-to-8 decoder using NAND gates.

Ans:

The figure below shows the truth table of 3-to-8 line decoder using NAND gates. Each output in the table gives a max term representation. At a given time only one output is low and all other outputs will be high. For example, when $A=B=1$ and $C=0$, then the output Y_6 is zero and all other outputs are high as shown in below figure.

[illegible]

From the above table, a 3-to-8 line decoder is designed by using three NAND gates and three NOT gates. NOT gates generate the complement of input while the NAND gates generate max terms of each output as shown in below figure.



6. How parity bit can be used for single-bit error detection? Explain with an example.

Ans:

Parity Bit Method : A parity bit is an extra bit included in binary message to make total number of 1's either odd or even. Parity word denotes number of 1's in a binary string. There are two parity system – even and odd parity checks.

There are two common types of parity: even parity and odd parity.

Even Parity: The total number of set (1) bits, including the parity bit, should be even.

Odd Parity: The total number of set (1) bits, including the parity bit, should be odd.

Example: A parity bit, added to data, counts set bits for even or odd parity.

Example: Data 1011 with even parity becomes 10110. If received data's parity doesn't match set bit count, a single-bit error is detected, e.g., 11110 indicates error.

7. What is a full adder circuit. Why it is used? Construct a full adder circuit using two half adder.

Ans:

A full adder circuit is a digital logic circuit that performs the arithmetic operation of adding three binary digits together. It takes three inputs: A, B, and C_{in} (carry-in), and produces two outputs: Sum (S) and C_{out} (carry-out). The full adder is used in digital circuits like CPUs to add binary numbers...

A full adder circuit is a digital logic component that adds three binary inputs (A, B, C_{in}) to produce a sum output (S) and a carry output (C_{out}). It's essential for arithmetic operations in CPUs and other digital systems... The logic diagram of the full adder using two half adders is shown in Figure-3:

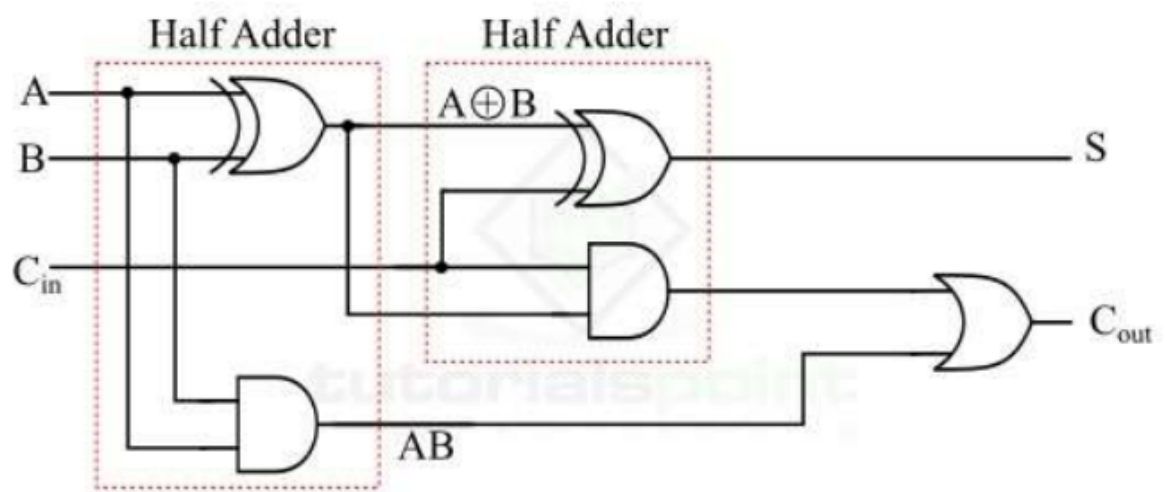


Figure 3 - Logic Diagram of Full Adder using Half Adder

8. Implement the function $Y=AB+CD$ using a 4-to-1 multiplexer. Show the truth table and circuit.

Step 1:

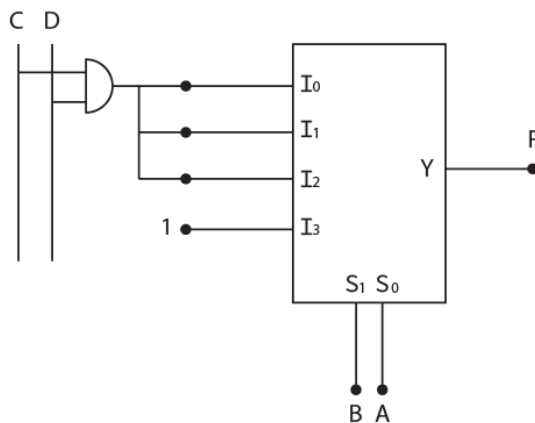
Make a truth table:

{Truth table is not mandatory ,if it is asked then implement this otherwise extra hassle}

A	B	C	D	$Y=AB+CD$
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Figure 0: truth table.

Step 2: implemented using 4:1 mux in following way.. This is an 4X1 MUX with inputs I_0, I_1, I_2, I_3 , Y as output and S_1, S_0 as selection lines. The output will depend upon the combination of S_1 & S_0 as shown in the truth table figure 2.



Step 3:

$$S_0 = A;$$

$$S_1 = B;$$

Step 4: create a truth table for selection lines.

S_1	S_0	Y
0	0	$I_0 = CD$
0	1	$I_1 = CD$
1	0	$I_2 = CD$
1	1	$I_3 = 1$

Figure 2: truth table

The 4-to-1 multiplexer can be used to implement the logic function $Y = AB + CD$. By selecting the appropriate inputs based on the truth table, the multiplexer produces the correct output value according to the given formula. This illustrates how multiplexers can be used to perform logic operations and implement complex functions using a combination of input signals and control bits.

9. Simplify this Boolean expression using Boolean logic, $(A+B'+C')(A+B'+C)(A+B+C')$

Ans:

Given Boolean expression: $(A + B' + C')(A + B' + C)(A + B + C')$

Now, using the distributive law, we can write

$$= [(A+B') + C'C](A + B + C')$$

Using $AA' = 0$, it becomes

$$[(A+B') + 0](A + B + C')$$

Since $A + 0 = A$,

$$= (A+B')(A + B + C')$$

$$= A + B'.(B+C')$$

Again, using distributive law

$$= A + B'B + B'C'$$

$$= A + 0 + B'C'$$

$$= A + B'C'.$$

10. What is the difference between encoder and multiplexer? Construct a full adder circuit with a 3-to-8 decoder.

Ans:

Encoder	Multiplexer
The encoder is a combinational circuit element that encodes a set of binary codes into another set of binary codes containing a smaller number of bits.	The multiplexer is a combinational circuit element that channels one of its many inputs to its only output depending on the selection inputs.
An encoder has n number of outputs if it has 2^n number of input pins.	A multiplexer has only one output pin irrespective of the number of input pins.
If an encoder has 2^n number of input pins.	A multiplexer has multiple numbers of input pins and the number of input pins is given by 2^n and it has only 1 output.
The encoder does not have any selection input lines.	If a multiplexer has 2^n number of input pins then it will have n number of selection lines.
The encoder is used to encode a particular set of binary sequences into another set binary sequence of a smaller number of bits.	The multiplexer is used to multiplex one of its input signals into its output depending on the selection input.

8-bit code to BCD, 16-bit code to 4-bit binary code, etc. are the examples of applications of the encoder.

Parallel to serial converter, digital switch, signal multiplexing, etc. are the examples of applications of the multiplexer.

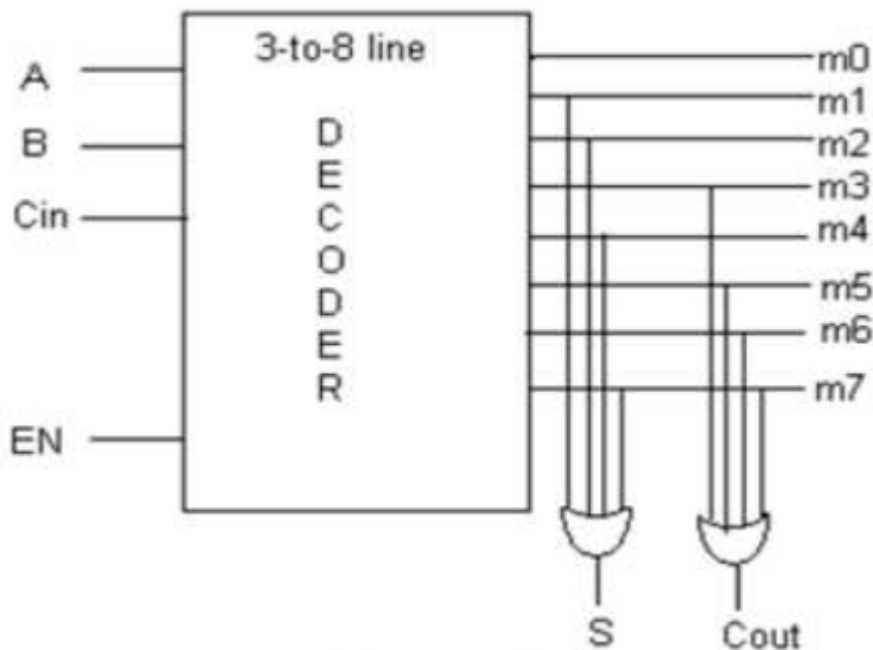


Fig1: Full Adder Implementation using 3:8 decoder

11. Define multiplexer and explain its role in digital systems. Implement $F(A,B,C,D) = \sum (1, 2, 4, 8, 9, 15)$ with an 8×1 MUX.

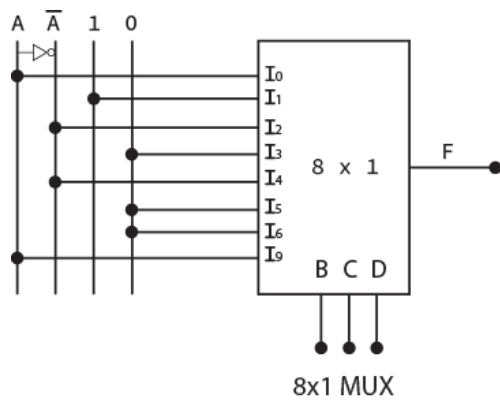
Part 1:

A multiplexer is a combinational circuit that has 2^n input lines and a single output line. There are n selection lines and 2^n input lines. So, there is a total of 2^n possible combinations of inputs. A multiplexer is also treated as Mux. Here's a concise list of the roles and applications of multiplexers:

- Data Routing
- Digital Data Transmission (Time-Division Multiplexing)
- Analog Signal Selection
- Address Decoding
- Display Multiplexing
- Data Selector (e.g., in ALUs)
- Testing and Debugging

Part 2:

A	B	C	D	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1



	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
A'	0	1	2	3	4	5	6	7
A	8	9	10	11	12	13	14	15
	A	1	A'	0	A'	0	0	A

NOTE: বোল্ড করে দেয়া নাম্বার গুলা গোল করে হবে। যেমন:

(2)

B	C	D	F
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

12. Define shift registers, ripple counter, discuss their application in digital systems.

Ans:

Shift Register is a group of flip flops used to store multiple bits of data. The bits stored in such registers can be made to move within the registers and in/out of the registers by applying clock pulses. An n-bit shift register can be formed by connecting n flip-flops where each flip-flop stores a single bit of data.

Applications of Shift Registers:

- The shift registers are used for temporary data storage.
- The shift registers are also used for data transfer and data manipulation.
- The serial-in serial-out and parallel-in parallel-out shift registers are used to produce time delay to digital circuits.

Ripple counter is a cascaded arrangement of flip-flops where the output of one flip-flop drives the clock input of the following flip-flop.

Applications of ripple Counters :

- They are used as frequency dividers, as divide by "N" counters.
- They are used for low noise emission and low power applications.
- They are used in designing asynchronous decade counters.
- It is also used in Ring counters and Johnson counters.
- Asynchronous counters are used in Mod N ripple counters. i.e. Mod 3, Mod 4, Mod 8, Mod 14, Mod 10 etc.

13. What are the advantages of synchronous counter over asynchronous counter? Illustrate with examples.

Ans:

1. Synchronous counters are easy to design since the same clock pulse is required for all flip-flops.
2. Synchronous counters have the same time delay throughout the circuit, thus by knowing the time delay of any flip-flop, we can predict the timing of output but in case of Asynchronous counters, time delay gets increased at each level of flip-flop.
3. Synchronous counters are fast while Asynchronous counters are slow.

Example 1: 4-Bit Binary Up Counter.

- **Consistent Timing:** All bits change together on clock cycles.
- **Reduced Glitches:** Synchronous operation minimises glitches.
- **Predictable Outputs:** Binary counting sequence is maintained.

Example 2: Cascade of Counters.

- **Easier Cascading:** Simple distribution of a single clock.
- **Consistent Timing:** Synchronised update for accurate counting.

14. Design a MOD-7 synchronous up counter with proper diagram.

Ans:

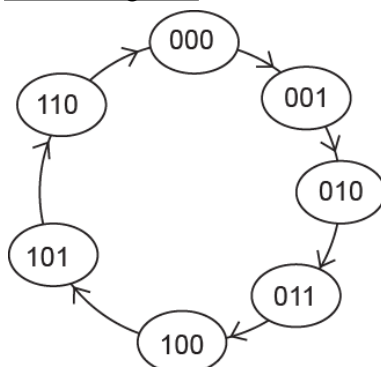
******NOTE: Truth table e j,k not needed ,korle valo**

Number of States=7

Range = 0 to n-1 = 0 to 6

Number of FF = 3 $[N \leq 2^n \Rightarrow \leq 2^3 \Rightarrow 7 \leq 8]$

State Diagram:



State Table:

Present State			Next State			I/Ps of FF					
Q_A	Q_B	Q_C	Q_{AH}	Q_{BH}	Q_{CH}	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	0	1	0	x	0	x	1	x
0	0	1	0	1	0	0	x	1	x	x	1
0	1	0	0	1	1	0	x	x	0	1	x
0	1	1	1	0	0	1	x	x	1	x	1
1	0	0	1	0	1	x	0	0	x	1	x
1	0	1	1	1	0	x	0	1	x	x	1
1	1	0	0	0	0	x	1	x	1	0	x
1	1	1	x	x	x	x	x	x	x	x	x

$$J_A = \sum m(3) + d(4, 5, 6, 7)$$

$$K_A = \sum m(6) + d(0, 1, 2, 3, 7)$$

$$J_B = \sum m(1, 5) + d(2, 3, 6, 7)$$

$$K_B = \sum m(3, 6) + d(0, 1, 4, 5, 7)$$

$$J_C = \sum m(0, 2, 4) + d(1, 3, 5, 7)$$

$$K_C = \sum m(1, 3, 5) + d(0, 2, 4, 6, 7)$$

	$\bar{Q}_B\bar{Q}_C$	$\bar{Q}_BQ_C$	Q_BQ_C	$Q_B\bar{Q}_C$
$\bar{Q}_A$			1	
Q_A	x	x	x	x

$$J_A = Q_BQ_C$$

	$\bar{Q}_B\bar{Q}_C$	$\bar{Q}_BQ_C$	Q_BQ_C	$Q_B\bar{Q}_C$
$\bar{Q}_A$	x	x	x	x
Q_A			x	1

$$K_A = Q_B$$

	$\bar{Q}_B\bar{Q}_C$	$\bar{Q}_BQ_C$	Q_BQ_C	$Q_B\bar{Q}_C$
$\bar{Q}_A$		1	x	x
Q_A		1	x	x

$$J_B = Q_C$$

	$\bar{Q}_B\bar{Q}_C$	$\bar{Q}_BQ_C$	Q_BQ_C	$Q_B\bar{Q}_C$
$\bar{Q}_A$	x	x	1	
Q_A	x	x	x	1

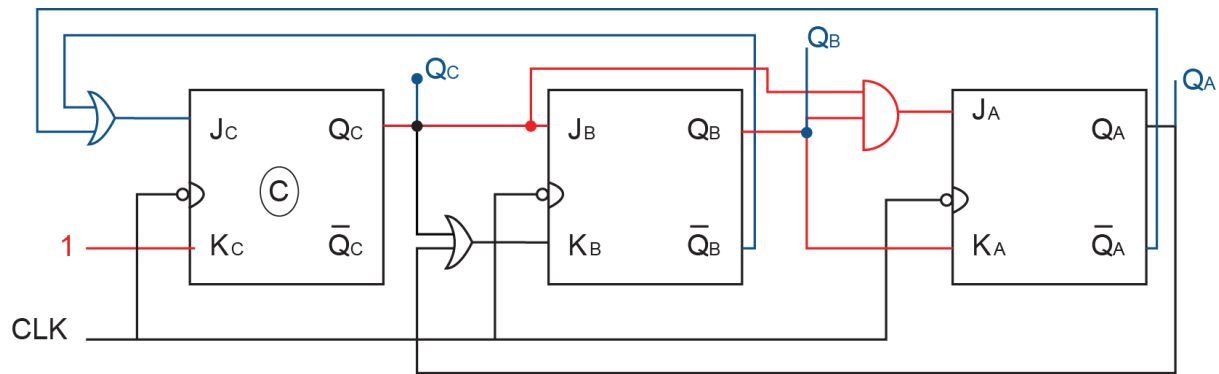
$$K_B = Q_C + Q_C$$

	$\bar{Q}_B\bar{Q}_C$	$\bar{Q}_BQ_C$	Q_BQ_C	$Q_B\bar{Q}_C$
$\bar{Q}_A$	1	x	x	1
Q_A	1	x	x	

$$J_C = Q_A + Q_B$$

	$\bar{Q}_B\bar{Q}_C$	$\bar{Q}_BQ_C$	Q_BQ_C	$Q_B\bar{Q}_C$
$\bar{Q}_A$	x	1	1	x
Q_A	x	1	x	x

$$K_C = 1$$



15. Write the difference between combinational and sequential circuits.

Ans:

Combinational circuits are defined as the time independent circuits which do not depend upon previous inputs to generate any output; they are termed as combinational circuits.

Sequential circuits are those which are dependent on clock cycles and depend on present as well as past inputs to generate any output.

Combinational Circuit –

{NB: first 1,2,4,5,8,9 is enough}

1. In this output depends only upon present input.
2. Speed is fast.
3. It is designed easy.
4. There is no feedback between input and output.
5. This is time independent.
6. Elementary building blocks: Logic gates
7. Used for arithmetic as well as boolean operations.
8. Combinational circuits don't have the capability to store any state.
9. As combinational circuits don't have clock, they don't require triggering.
10. These circuits do not have any memory element.
11. It is easy to use and handle.

Examples – Encoder, Decoder, Multiplexer, Demultiplexer Block Diagram –

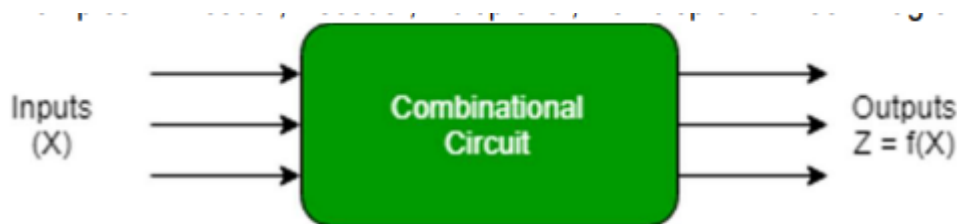


Figure: Combinational Circuits

Sequential Circuit –

{NB: 1,2,4,6,8,9}

1. In this output depends upon present as well as past input.
2. Speed is slow.
3. It is designed tough as compared to combinational circuits.
4. There exists a feedback path between input and output.
5. This is time dependent.
6. Elementary building blocks: Flip-flops

7. Mainly used for storing data.
8. Sequential circuits have capability to store any state or to retain earlier state.
9. As sequential circuits are clock dependent they need triggering.
10. These circuits have memory element.
11. It is not easy to use and handle.

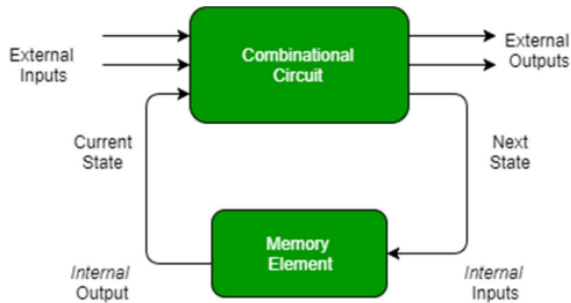


Figure: Sequential Circuit

16. Design a D-type flip-flop using NAND gates. Provide the truth table and circuit diagram.

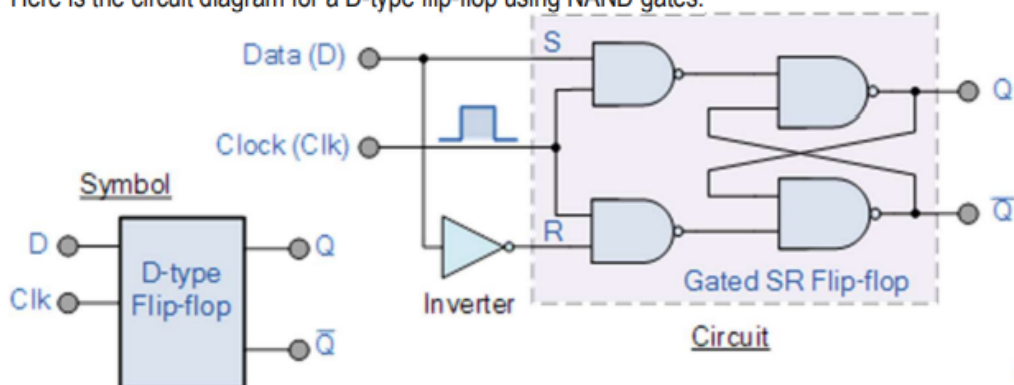
Ans:

A D-type flip-flop can be designed using NAND gates. Here is the truth table for a D-type flip-flop:

Clk	D	Q	Q	Description
↓ » 0	X	Q	Q	Memory no change
↑ » 1	0	0	1	Reset Q » 0
↑ » 1	1	1	0	Set Q » 1

Here is the circuit diagram for a D-type flip-flop using NAND gates:

here is the circuit diagram for a D-type flip-flop using NAND gates.



We remember that a simple SR flip-flop requires two inputs, one to “SET” the output and one to “RESET” the output. By connecting an inverter (NOT gate) to the SR flip-flop we can “SET” and “RESET” the flip-flop using just one input as now the two input signals are complements of each other. This complement avoids the ambiguity inherent in the SR latch when both inputs are LOW, since that state is no longer possible.

17. Convert a JK flip-flop into D flip-flop. Explain with circuit diagram.

Ans:

A JK flip-flop can be converted into a D flip-flop by using the following steps:

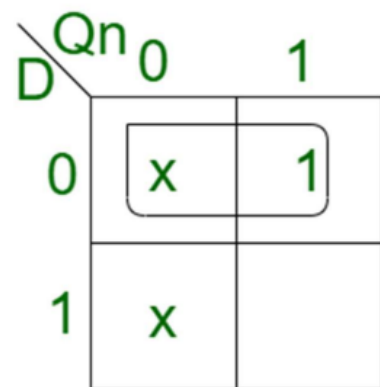
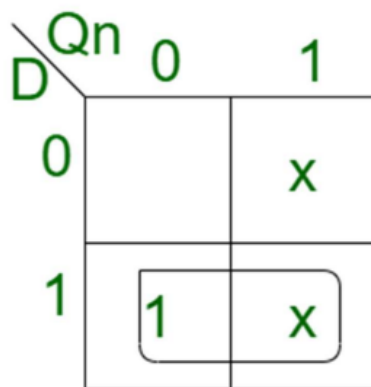
- Step-1:

We construct the characteristic table of D flip-flop and excitation table of JK flip-flop.

D	Q_n	Q_{n+1}	J	K
0	0	0	0	x
0	1	0	x	1
1	0	1	1	x
1	1	1	x	0

- Step-2:

Using the K-map we find the boolean expression of J and K in terms of D.

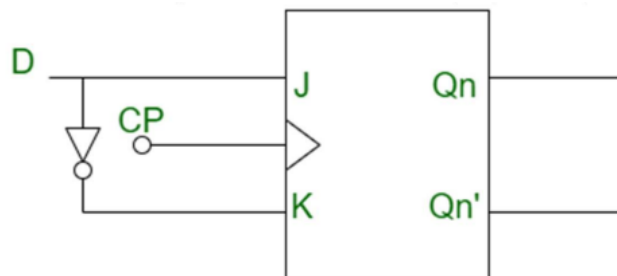


$$J = D$$

$$K = D'$$

- Step-3:

We construct the circuit diagram of the conversion of JK flip-flop into D flip-flop.



We can see that there is only one input from the D where the direct input is going into J input and inverse or altered input is going to K input ,

18. Discuss the advantages and applications of field programmable gate arrays (FPGAs) in digital design and its architecture. (PAL) and programmable logic array (PLA).

Ans:

Field Programmable Gate Arrays (FPGAs) are integrated circuits that consist of internal hardware blocks with user-programmable interconnects to customize operation for a specific application. The interconnects can readily be reprogrammed, allowing an FPGA to accommodate changes to a design or even support a new application during the lifetime of the part. FPGAs have several advantages over other digital design technologies. They are highly flexible and can be programmed to perform any digital function. They are also highly parallel and can perform multiple operations simultaneously. Programmable Array Logic (PAL) and Programmable Logic Array (PLA) are two types of programmable logic devices that were used before FPGAs became popular. PALs are programmable logic devices that have a fixed OR array and a programmable AND array. PLAs are programmable logic devices that have both programmable AND and OR arrays.

19. Explain the purpose and functioning of programmable array logic.

Ans:

Programmable Logic Array (PLA) is a fixed architecture logic device with programmable AND gates followed by programmable OR gates. PLA is basically a type of programmable logic device used to build a reconfigurable digital circuit. A PLA is a fixed architecture logic device with programmable AND gates followed by programmable OR gates. It is used to build a reconfigurable digital circuit. A programmable logic array (PLA) is a type of logic device that can be programmed to implement various kinds of combinational logic circuits. A programmable logic array is designed such that a number of different logical functions can be combined as a sum-of-product or product-of-sum form. A PLA having N input buffers and M output buffers consists of $2N$ AND gates and M OR gates, each with programmable inputs from all of the AND gates. PLAs have widely been acknowledged as compact and space-efficient solutions for many complicated circuits, especially in feedback and control systems where a number of factor variables must be involved for efficient functioning of the system.

20. Implement the Boolean function using PAL.

Ans:

$$Y_1 = \sum m(2, 3, 5, 7)$$

$$Y_2 = \sum m(3, 6)$$

Implement the given function by using PAL:

Following Truth table will be helpful in understanding function on number of inputs:

Step-1:

A	B	C	Y_1	Y_2
0	0	0	0	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	1
1	0	0	0	0
1	0	1	1	0
1	1	0	0	1
1	1	1	1	0

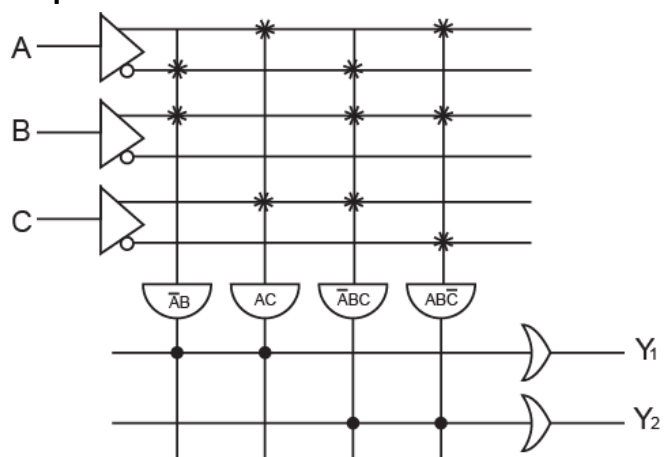
Step-2:

$$\therefore Y_1 = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + ABC$$

$$Y_1 = \bar{A}B + AC$$

$$Y_2 = \bar{A}BC + AB\bar{C}$$

Step-3:



21. what is hardware description language what are the requirement of a good sdl describe the abstraction level of very low HDL.

Ans:

A hardware description language (HDL) is a programming language used to describe the structure and behavior of electronic circuits. HDLs are used to design digital circuits, such as microprocessors, digital signal processors, and field-programmable gate arrays (FPGAs).

The requirements of a good HDL are:

- Expressiveness: The HDL should be able to describe a wide variety of electronic circuits.
- Efficiency: The HDL should be efficient in terms of both time and space.
- Portability: The HDL should be portable to different hardware platforms.
- Verifiability: The HDL should be able to be verified to ensure that it accurately describes the desired circuit.
- Standardization: The HDL should be standardized so that it can be used by different designers and tools.

The abstraction level of a hardware description language refers to the level of detail at which it describes a circuit. Very low-level HDLs (VHLs) describe the circuit at a very detailed level, down to the individual transistors. This makes them very accurate, but also very difficult to use. (এই অঙ্কি মেইন এন্সার, পড়ের গুলা এক্সট্রা)

Some examples of VHLs are:

- Verilog-XL
- VHDL-AMS
- SystemC

Higher-level HDLs describe the circuit at a more abstract level, using concepts such as registers, memories, and arithmetic units. This makes them easier to use, but less accurate.

Some examples of higher-level HDLs are:

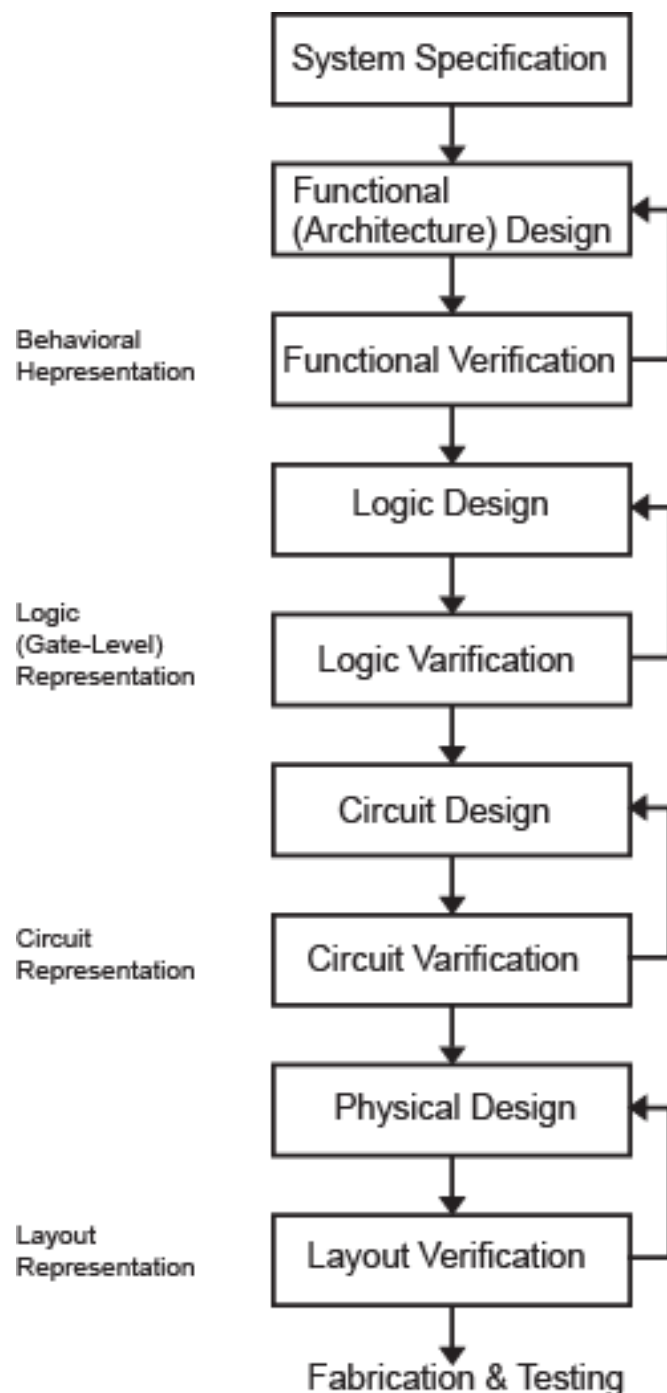
- Verilog
- VHDL

The choice of HDL depends on the specific application. VHLS are typically used for designing complex circuits where accuracy is critical. Higher-level HDLS are typically used for designing simpler circuits or for rapid prototyping.

I hope this explanation is helpful. Let me know if you have any other questions.

22. Draw the HDL Design Flow.

Ans:



23. Write a VHDL code to implement a 4bit full adder.

Ans:

The VHDL code to implement a 4bit full adder is given below:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity full_adder_4bit is
    Port ( A : in  STD_LOGIC_VECTOR(3 downto 0);
          B : in  STD_LOGIC_VECTOR(3 downto 0);
          Cin : in  STD_LOGIC;
          Sum : out STD_LOGIC_VECTOR(3 downto 0);
          Cout : out STD_LOGIC);
end full_adder_4bit;

architecture Behavioral of full_adder_4bit is
    signal temp_sum : STD_LOGIC_VECTOR(3 downto 0);
begin
    process(A, B, Cin)
    begin
        for i in 0 to 3 loop
            temp_sum(i) <= A(i) XOR B(i) XOR Cin;
            if i = 0 then
                Cout <= (A(i) AND B(i)) OR (Cin AND (A(i) XOR B(i)));
            else
                Cout <= (temp_sum(i-1) AND ((A(i) XOR B(i)) OR Cin)) OR ((A(i) AND B(i)) OR
(Cin AND (A(i) XOR B(i))));
            end if;
        end loop;
    end process;

    Sum <= temp_sum;

end Behavioral;
```